04 — Retailer Data, Scheduler and Notifications Guide

Audience: Member 4 (Retailer Data, Scheduler and Notifications Developer). **Status:** Implementation handbook. Every schema, endpoint, field name, environment variable, provider interface shape, and alert-trigger rule in this document must match `docs/01-MASTER-PROJECT-DEVELOPMENT-GUIDE.md` ("Document 1") exactly. If anything here ever appears to disagree with Document 1, Document 1 is correct — stop and reconcile with Member 1 (the team lead) before writing code.

Task ID scheme used throughout this document: `DATA-01` through `DATA-07` for retailer-provider and scheduler tasks, `NOTIF-01` through `NOTIF-03` for notification tasks. These IDs are unique to this document; they do not collide with `BE-xx` (Document 3), `FE-xx` (Document 2), or `DEP-xx` (Document 5) IDs used elsewhere.

---

1. Scope

Member 4 owns:

- `server/src/providers/` — `index.js` (the dispatcher), `fixture.js` + `fixtureData.json`, `flipkart.js`, and (as sample-only, never a live code path) `amazon.js`.
- `server/src/jobs/` — **local-dev convenience only.** Production scheduling is GitHub Actions (`.github/workflows/monitor.yml`), which is wired up together with Member 1/Document 5 because it needs the deployed backend URL. Anything in `jobs/` is a script you run yourself from PowerShell during development; nothing in `jobs/` runs automatically in production.
- `server/src/services/notifications/` — `email.js`, `telegram.js`, and the notification-event dedup/retry logic that writes to the `notificationevents` collection (Document 1, Section 5.7 — Member 4 owns this collection's write logic even though the Mongoose model *file* lives in `server/src/models/`, which Member 3 creates and maintains).

Member 4 **never edits** `server/src/services/priceComparison.js`, `server/src/services/priceMonitor.js`, or `server/src/services/alertEvaluator.js` without coordinating with Member 1 first (Document 1, Section 14). Those files call *into* the provider dispatcher and the notification services this document builds — Member 4's job is to make sure that interface is solid and well-tested, not to own the orchestration logic itself.

A quick mental model for the whole document: **Member 4 builds the things that reach outside the project** — out to Flipkart (if and when approved), out to Brevo, out to Telegram — and makes sure every one of those outside calls comes back as a clean, predictable result no matter what actually happened on the other end, including when it fails.

---

2. Day 1: retailer-access investigation — do this FIRST, before writing any code

Document 1, Section 10 already records the **verified findings** from this investigation (dated 2026-10-13). This section tells you how to *act* on those findings today, and what to actually go register for, before touching a single line of provider code.

2.1 What's already verified — read this before you do anything else

- **Amazon India Creators API is excluded, full stop.** Document 1 Section 10.2 verified, directly from Amazon's own docs, that the Creators API requires **10 qualifying sales within the past 30 days** before an account can even reach the Product Advertising API. A brand-new team with a brand-new Associates account has zero sales history and cannot satisfy this from a standing start in a 7-day project. This isn't a timing problem you can work around by applying earlier — it's a structural eligibility gate. **Do not attempt to register for or call the Amazon Creators API at any point in this project.** Amazon only ever appears in PriceRadar as a second, clearly-labeled `isSample: true` fixture offer (Section 6 below).
- **Flipkart Affiliate API approval timing is unverified and not instant.** Document 1 Section 10.1 verified, from Flipkart's own docs, that the Product/Offer/Report APIs are available only to *registered, approved* affiliates — there is no self-serve or unauthenticated tier. Third-party sources researched alongside the official docs report approval windows from a few days to a few weeks, but Flipkart's own documentation does not publish an SLA, so treat "a few days to a few weeks" as directionally true but **not a promise**.

2.2 Register for the Flipkart Affiliate Program today

1. Go to **https://affiliate.flipkart.com/registerme**.
2. The registration form asks for a "site" (the affiliate program is built around linking from a website or app you run). **You very likely do not have a live deployed PriceRadar URL yet on Day 1** — that's fine and expected. Use whichever of these is available at the moment you register, in this order of preference:
   - The team's GitHub repository URL (`https://github.com/<org>/<repo>`), if the repo is public.
   - A placeholder note that you intend to use your deployed PriceRadar frontend once it is live (some affiliate programs accept a "site in development" description — fill in what the form allows honestly, do not invent a URL that doesn't exist).
3. Fill in the registration details accurately. Third-party guidance alongside the official docs suggests KYC-style details (PAN, etc.) may be requested — Flipkart's own policy requires accurate registration information, so do not fabricate anything to get through the form faster.
4. Submit, and **record the exact date and time you submitted** in the findings note (Section 3 below). This is your evidence for whether "a few days to a few weeks" turns out to be accurate for your application.
5. **Be honest with the team that this registration may need to be redone or updated** once Document 5's real deployment lands and the team has an actual public URL — if Flipkart's review process asks for a working site link later, the team will need to supply the real deployed frontend URL

at that point. Flag this explicitly in the findings note so it isn't a surprise on Day 6.

2.3 Do not wait for approval to start Day 2

This is the single most important instruction in this section: **submit the Flipkart registration today, and then immediately move on to building the fixture provider (Section 6) regardless of what happens with that application.** Approval might arrive on Day 2, Day 6, or not at all within the 7-day window — Document 1's whole provider strategy (Section 10.3–10.6) is built around the fixture provider being the thing the entire team's Day 2–7 work actually depends on. Flipkart integration is a bonus if it lands in time (Section 7 below shows you how to wire it in when and if it does), never a blocker.

---

3. Findings note template

Create `docs/day1-retailer-findings.md` in the repo and fill this in today. This satisfies the brief's requirement (MASTER-PROMPT-SOURCE.md, Section 22): label what's unverified, explain how to test it, state the impact if it fails, and provide a fallback — all already decided, not left as an open question.

```
# Day 1 Retailer Data Source Findings

  **Investigated by:** Member 4
  **Date:** 2026-10-13 (or your actual Day 1 date)

  ## Source 1: Flipkart Affiliate API

  - Official docs reviewed: https://affiliate.flipkart.com/api-docs/af_overview.html,
    https://affiliate.flipkart.com/api-docs/af_register.html
  - Finding: Product/Offer/Report APIs require a registered, approved affiliate accoun
  t.
    No self-serve/unauthenticated tier exists.
  - Registration status: Submitted at affiliate.flipkart.com/registerme on <date/time>.
    Site URL supplied: <GitHub repo URL / placeholder note — see Section 2.2>.
  - Approval timeline: UNVERIFIED — Flipkart's own docs do not publish an SLA.
    Third-party reports suggest days to weeks. Will update this file the moment
    approval (or rejection) actually arrives, with the real date.
  - How the team will test it if/when approval arrives: make one real request from
    `providers/flipkart.js` against a known product, confirm a real price comes back,
    only then follow the switch-over checklist in Document 4 Section 7.4.
  - Impact if it never arrives within 7 days: none to the demo's viability — the
    entire system is built and tested against the fixture provider from Day 1.
    Flipkart data is a bonus, not a requirement for "done" (Document 1, Section 10.6).
  - Fallback: fixture provider (always available, Section 6 of Document 4).

  ## Source 2: Amazon India Creators API

  - Official docs reviewed: https://affiliate-program.amazon.in/creatorsapi/docs/en-us/
  introduction
  - Finding: VERIFIED — requires 10 qualifying sales within the past 30 days before
    PA API access is granted. A new team/account cannot meet this from zero inside
    7 days. This is a hard eligibility gate, not a timing issue.
  - Decision: EXCLUDED from any real integration attempt. No code path ever calls
    this API. Amazon appears in the UI only as `isSample: true` fixture data,
    labeled "Sample data" everywhere it's shown.

  ## Decision

  Build and test the entire system against `providers/fixture.js` starting Day 2,
  independent of Flipkart's approval status. Swap in real Flipkart data only if and
  when approval is verified by an actual successful API response — never on an
  assumption.
```

Keep this file up to date — when (if) the Flipkart approval email actually arrives, come back and fill in the real date and outcome rather than leaving it stale.

---

4. Project setup for this member's area

4.1 Confirm Node and the shared scaffold exist

```
node --version
  npm --version
```

You need Node.js v20 LTS or higher, same as the rest of the team (Document 1, Section 3). If `server/` doesn't exist yet, Member 3 hasn't finished Day 1 scaffolding — coordinate before proceeding; don't create a competing `server/` folder yourself.

4.2 Check before installing — don't blindly reinstall

Member 3's Day 1 scaffold likely already ran `npm install` for the shared backend dependencies. Check what's already there before adding anything:

```
cd D:\Projects\PriceRadar\server
  Get-Content package.json
```

Look in `dependencies` for `axios` and `cheerio`. If both are already listed, skip straight to `npm install` (to make sure your own `node_modules/` is populated) and do not run `npm install axios cheerio` again — doing so is harmless but unnecessary, and if either package is *missing*, add only the missing one(s):

```
npm install
  # Only if axios and/or cheerio are NOT already in package.json:
  npm install axios cheerio
```

- **axios** — the HTTP client this entire project uses consistently (Document 1, Section 3). You'll use it for Flipkart requests, Brevo requests, and Telegram Bot API requests.
- **cheerio** — an HTML-parsing library, included in the stack for the hypothetical case where a provider needs to parse an HTML response rather than JSON. The Flipkart Affiliate API (Section 7) returns JSON, so you likely won't need Cheerio for Flipkart specifically, but it stays in the stack per Document 1, Section 3 in case a future provider needs it. Do not add any other HTML-scraping package — Document 1, Section 1.2 explicitly excludes browser-based scraping workarounds.

Do not add any package not listed in Document 1, Section 3's technology table without updating that document first.

4.3 Create your folders if Member 3 hasn't yet

```
cd D:\Projects\PriceRadar\server\src
  New-Item -ItemType Directory -Force -Path providers
  New-Item -ItemType Directory -Force -Path jobs
  New-Item -ItemType Directory -Force -Path services\notifications
```

`-Force` makes these safe to run even if the folders already exist — it won't overwrite anything inside them.

4.4 Fill in your fields in `server/.env`

Member 3 creates `server/.env` (from `server/.env.example`) as part of the shared Day 1 scaffold. Open `server/.env` and fill in (or leave blank for now, per the notes) the variables this document owns, exactly as named in Document 1, Section 7.1:

```
FLIPKART_AFFILIATE_ID=
  FLIPKART_AFFILIATE_TOKEN=
  BREVO_API_KEY=
  EMAIL_FROM=
  TELEGRAM_BOT_TOKEN=
  MONITOR_TRIGGER_SECRET=
  MONITORING_ENABLED=true
```

- `FLIPKART_AFFILIATE_ID` / `FLIPKART_AFFILIATE_TOKEN` — leave **blank** until (if) Flipkart approval actually arrives with real credentials. The fixture provider doesn't need them. Never put a placeholder string here that looks like a real credential.
- `BREVO_API_KEY` — fill in once you've created the shared team Brevo account (Section 11).
- `EMAIL_FROM` — the verified sender address from Brevo (Section 11).
- `TELEGRAM_BOT_TOKEN` — fill in once you've created the bot via @BotFather (Section 12).
- `MONITOR_TRIGGER_SECRET` — generate this the same way as `JWT_SECRET` (Document 1, Section 7.1): `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`. This is shared with Member 1/Document 5, since it also needs to be set as a GitHub Actions secret and on the deployed backend.
- `MONITORING_ENABLED` — `true` for normal development. Setting it to `false` is a safety valve Member 1's `priceMonitor.js` checks before doing any provider work at all.

Also update `server/.env.example` (the **committed** file, placeholders only) so teammates cloning the repo know these variables exist:

```
FLIPKART_AFFILIATE_ID=
  FLIPKART_AFFILIATE_TOKEN=
  BREVO_API_KEY=your-brevo-api-key-here
  EMAIL_FROM=alerts@yourteamdomain.example
  TELEGRAM_BOT_TOKEN=your-telegram-bot-token-here
  MONITOR_TRIGGER_SECRET=generate-with-node-crypto-randomBytes
  MONITORING_ENABLED=true
```

Never commit `server/.env` itself — only `server/.env.example`. Double-check `server/.gitignore` lists `.env` before your first commit.

---

5. The provider interface (`server/src/providers/index.js`)

5.1 Why a shared interface matters, in plain terms

Imagine `priceMonitor.js` (Member 1's file) had to know, for every single offer it checks, "if this is a Flipkart offer, call it this way; if it's a fixture offer, call it that other way; if it's an Amazon offer, do a third thing." That would mean every time Member 4 changes how a provider works internally, Member 1's monitoring code would also need to change. Two people would be editing around the same problem from two different files, which is exactly the kind of coupling Document 1 is trying to avoid (Section 1.3: "no feature is implemented in a way that only one team member understands").

Instead, every provider file — `fixture.js`, `flipkart.js`, eventually anything else — promises to answer the exact same question the exact same way: *"Here's an offer I want a fresh price for. Either tell me the fresh data, or tell me clearly that it failed."* `priceMonitor.js` only ever talks to **one** function, the dispatcher's `fetchOffer(offer)`, and never needs to know or care which real provider is underneath. This is sometimes called the "Strategy pattern" or "Adapter pattern" in software design textbooks — the plain-English version is just: **one shared shape in, one shared shape out, no matter which provider is actually doing the work.**

5.2 The exact `NormalizedResult` shape (Document 1, Section 10.5 — do not deviate)

```
// Every provider — real or fixture — returns exactly this shape.
  {
    provider: "flipkart" | "amazon",
    externalProductId: string,
    title: string,
    brand: string | null,
    model: string | null,
    variant: string | null,
    priceInPaise: number,       // integer, only meaningful when success === true
    currency: "INR",
    availability: "in_stock" | "out_of_stock" | "unknown",
    productUrl: string,
    imageUrl: string | null,
    isSample: boolean,
    fetchedAt: string,          // ISO 8601 — set by the dispatcher, not by the individ
  ual provider file
    success: boolean,
    errorMessage: string | null // present when success === false, otherwise null
  }
```

The single most important rule for every provider function: it never throws. A network timeout, a 404, a malformed response, a rate limit, a missing field — all of these are caught *inside* the provider and turned into `{ success: false, errorMessage: "..." }`. This is what lets `priceMonitor.js` call every provider the exact same way without wrapping every single call in its own `try/catch` — if a provider ever throws instead of returning a failure object, it breaks that assumption for the whole monitoring cycle (see Section 15, "a provider function accidentally throwing").

5.3 The dispatcher

```
// server/src/providers/index.js
  //
  // This is the ONE place priceMonitor.js (Member 1) calls into. It decides which
  // underlying provider file actually handles a given offer, and guarantees every
  // caller gets back the same NormalizedResult shape (Document 1, Section 10.5)
  // no matter which provider answered.

  const fixtureProvider = require('./fixture');
  const flipkartProvider = require('./flipkart');

  /**
   * Fetch a fresh price/availability snapshot for one Offer.
   *
   * @param {object} offer - a lean object or Mongoose doc with at least:
   *   { provider: "flipkart"|"amazon", externalProductId: string, isSample: boolean }
   * @returns {Promise<NormalizedResult>} - always resolves, never rejects/throws.
   */
  async function fetchOffer(offer) {
    let result;

    if (offer.isSample) {
      // Every fixture/sample offer — whether labeled "flipkart" or "amazon" — is
      // served by the fixture provider. isSample, not the provider label, decides
      // routing here, because a "flipkart"-labeled offer can still be sample data
      // until Flipkart approval is verified (Document 1, Section 10.4).
      result = await fixtureProvider.fetchOffer(offer);
    } else if (offer.provider === 'flipkart') {
      // NOT WIRED IN BY DEFAULT. See providers/flipkart.js and the switch-over
      // checklist in Document 4, Section 7.4. Until that checklist has been
      // completed and verified, this branch is intentionally routed back to the
      // fixture provider so the system never silently calls a real API with
      // unverified or missing credentials.
      result = await fixtureProvider.fetchOffer(offer);
    } else if (offer.provider === 'amazon') {
      // Amazon NEVER has a live code path (Document 1, Section 10.2). Any
      // non-sample Amazon offer is a data bug, not something to fetch live.
      result = {
        success: false,
        errorMessage: 'Amazon has no live provider in this project — Amazon offers must
  always be isSample: true.',
      };
```

```
  } else {
    result = {
      success: false,
      errorMessage: `No provider configured for provider="${offer.provider}"`,
    };
  }

  // fetchedAt is set here, by the dispatcher, not by the individual provider
  // file — this keeps "when did we actually attempt this" consistent and in
  // one place, rather than trusting each provider to timestamp itself correctly.
  return {
    ...result,
    fetchedAt: new Date().toISOString(),
  };
}

module.exports = { fetchOffer };
```

Notice the dispatcher itself never throws either — even the "no provider configured" and "Amazon has no live path" cases return a proper `{ success: false, errorMessage }` object, keeping the promise consistent all the way up to `priceMonitor.js`.

---

6. The fixture provider (`server/src/providers/fixture.js` + `fixtureData.json`)

This is the provider the entire project runs on from Day 2 onward, independent of whatever happens with Flipkart (Section 2.3). Treat it as first-class code, not a placeholder.

6.1 `server/src/providers/fixtureData.json`

Six realistic Indian e-commerce products, each with a `flipkart`-labeled and an `amazon`-labeled offer — both `isSample: true`, matching Document 1, Section 10.3/10.4 exactly. Prices are in **integer paise** (Document 1, Section 5 precision policy). One offer (`FLP-NOISE-PULSE2-BLK`) is marked `forceFail: true` specifically so the team can test the failure path (ACC-06) on demand — see Section 6.4 for how to use it.

```
[
    {
      "canonicalKey": "samsung-galaxy-m14-5g-128gb-icy-silver",
      "title": "Samsung Galaxy M14 5G",
      "brand": "Samsung",
      "model": "Galaxy M14 5G",
      "variant": "128GB, Icy Silver",
      "imageUrl": "https://images.priceradar.sample/samsung-m14-5g.jpg",
      "offers": [
        {
          "provider": "flipkart",
          "externalProductId": "FLP-SGM14-128-SLV",
          "title": "SAMSUNG Galaxy M14 5G (Icy Silver, 128 GB)",
          "priceInPaise": 1349900,
          "currency": "INR",
          "availability": "in_stock",
          "productUrl": "https://www.flipkart.com/samsung-galaxy-m14-5g-icy-silver-128-
  gb/p/sample-sgm14",
          "forceFail": false
        },
        {
          "provider": "amazon",
          "externalProductId": "AMZ-SGM14-128-SLV",
          "title": "Samsung Galaxy M14 5G (Icy Silver, 128GB)",
          "priceInPaise": 1369900,
          "currency": "INR",
          "availability": "in_stock",
          "productUrl": "https://www.amazon.in/dp/sample-SGM14SLV",
          "forceFail": false
        }
      ]
    },
    {
      "canonicalKey": "redmi-note-13-5g-128gb-starburst-white",
      "title": "Redmi Note 13 5G",
      "brand": "Redmi",
      "model": "Note 13 5G",
      "variant": "128GB, Starburst White",
      "imageUrl": "https://images.priceradar.sample/redmi-note-13-5g.jpg",
      "offers": [
        {
          "provider": "flipkart",
```

```
        "externalProductId": "FLP-RDMN13-128-WHT",
        "title": "Redmi Note 13 5G (Starburst White, 128 GB)",
        "priceInPaise": 1699900,
        "currency": "INR",
        "availability": "in_stock",
        "productUrl": "https://www.flipkart.com/redmi-note-13-5g-starburst-white-128-gb/p/sample-rdmn13",
        "forceFail": false
      },
      {
        "provider": "amazon",
        "externalProductId": "AMZ-RDMN13-128-WHT",
        "title": "Redmi Note 13 5G (Starburst White, 128GB)",
        "priceInPaise": 1719900,
        "currency": "INR",
        "availability": "in_stock",
        "productUrl": "https://www.amazon.in/dp/sample-RDMN13WHT",
        "forceFail": false
      }
    ]
  },
  {
    "canonicalKey": "boat-airdopes-141-black",
    "title": "boAt Airdopes 141",
    "brand": "boAt",
    "model": "Airdopes 141",
    "variant": "Black",
    "imageUrl": "https://images.priceradar.sample/boat-airdopes-141.jpg",
    "offers": [
      {
        "provider": "flipkart",
        "externalProductId": "FLP-BOAT141-BLK",
        "title": "boAt Airdopes 141 Bluetooth Truly Wireless Earbuds",
        "priceInPaise": 129900,
        "currency": "INR",
        "availability": "in_stock",
        "productUrl": "https://www.flipkart.com/boat-airdopes-141/p/sample-boat141",
        "forceFail": false
      },
      {
        "provider": "amazon",
        "externalProductId": "AMZ-BOAT141-BLK",
        "title": "boAt Airdopes 141 TWS Earbuds, Black",
```

```
        "priceInPaise": 134900,
        "currency": "INR",
        "availability": "in_stock",
        "productUrl": "https://www.amazon.in/dp/sample-BOAT141BLK",
        "forceFail": false
      }
    ]
  },
  {
    "canonicalKey": "noise-colorfit-pulse-2-black",
    "title": "Noise ColorFit Pulse 2",
    "brand": "Noise",
    "model": "ColorFit Pulse 2",
    "variant": "Jet Black",
    "imageUrl": "https://images.priceradar.sample/noise-colorfit-pulse-2.jpg",
    "offers": [
      {
        "provider": "flipkart",
        "externalProductId": "FLP-NOISE-PULSE2-BLK",
        "title": "Noise ColorFit Pulse 2 Smart Watch, Jet Black",
        "priceInPaise": 179900,
        "currency": "INR",
        "availability": "in_stock",
        "productUrl": "https://www.flipkart.com/noise-colorfit-pulse-2/p/sample-noise
pulse2",
        "forceFail": true
      },
      {
        "provider": "amazon",
        "externalProductId": "AMZ-NOISE-PULSE2-BLK",
        "title": "Noise ColorFit Pulse 2 Smartwatch, Jet Black",
        "priceInPaise": 184900,
        "currency": "INR",
        "availability": "in_stock",
        "productUrl": "https://www.amazon.in/dp/sample-NOISEPULSE2",
        "forceFail": false
      }
    ]
  },
  {
    "canonicalKey": "mi-power-bank-3i-20000mah",
    "title": "Mi Power Bank 3i",
    "brand": "Mi",
```

```
    "model": "Power Bank 3i",
    "variant": "20000mAh, Black",
    "imageUrl": "https://images.priceradar.sample/mi-power-bank-3i.jpg",
    "offers": [
      {
        "provider": "flipkart",
        "externalProductId": "FLP-MIPB3I-20K-BLK",
        "title": "Mi Power Bank 3i 20000 mAh",
        "priceInPaise": 169900,
        "currency": "INR",
        "availability": "in_stock",
        "productUrl": "https://www.flipkart.com/mi-power-bank-3i-20000-mah/p/sample-m
ipb3i",
        "forceFail": false
      },
      {
        "provider": "amazon",
        "externalProductId": "AMZ-MIPB3I-20K-BLK",
        "title": "Mi 20000mAh Power Bank 3i",
        "priceInPaise": 174900,
        "currency": "INR",
        "availability": "out_of_stock",
        "productUrl": "https://www.amazon.in/dp/sample-MIPB3I20K",
        "forceFail": false
      }
    ]
  },
  {
    "canonicalKey": "hp-15s-ryzen5-8gb-512gb",
    "title": "HP 15s Laptop",
    "brand": "HP",
    "model": "15s",
    "variant": "Ryzen 5, 8GB RAM, 512GB SSD",
    "imageUrl": "https://images.priceradar.sample/hp-15s-ryzen5.jpg",
    "offers": [
      {
        "provider": "flipkart",
        "externalProductId": "FLP-HP15S-R5-8-512",
        "title": "HP 15s Ryzen 5 Hexa Core 5500U - (8 GB/512 GB SSD)",
        "priceInPaise": 4199900,
        "currency": "INR",
        "availability": "in_stock",
        "productUrl": "https://www.flipkart.com/hp-15s-ryzen5-8gb-512gb/p/sample-hp15
```

```
s",
        "forceFail": false
      },
      {
        "provider": "amazon",
        "externalProductId": "AMZ-HP15S-R5-8-512",
        "title": "HP 15s, AMD Ryzen 5, 8GB RAM, 512GB SSD",
        "priceInPaise": 4299900,
        "currency": "INR",
        "availability": "in_stock",
        "productUrl": "https://www.amazon.in/dp/sample-HP15SR5",
        "forceFail": false
      }
    ]
  }
]
```

6.2 `server/src/providers/fixture.js`

```
// server/src/providers/fixture.js
  //
  // The guaranteed fallback provider (Document 1, Section 10.3). Every offer this
  // project runs on until/unless Flipkart approval is verified goes through here.
  // Reads server/src/providers/fixtureData.json and simulates a real network call
  // — including the ability to deliberately fail one entry on demand, so the team
  // can test the failure path (ACC-06) without needing a real provider outage.

  const fixtureData = require('./fixtureData.json');

  // Flatten the product-grouped JSON into a single lookup keyed by
  // "provider:externalProductId", which is exactly how an Offer document
  // identifies which fixture entry it corresponds to.
  const offersByKey = new Map();
  for (const product of fixtureData) {
    for (const offer of product.offers) {
      const key = `${offer.provider}:${offer.externalProductId}`;
      offersByKey.set(key, { ...offer, product });
    }
  }

  function sleep(ms) {
    return new Promise((resolve) => setTimeout(resolve, ms));
  }

  /**
   * @param {object} offer - { provider, externalProductId, ... }
   * @returns {Promise<object>} a partial NormalizedResult (fetchedAt is added
   *   by the dispatcher in providers/index.js, not here).
   */
  async function fetchOffer(offer) {
    // Simulate a small, realistic network delay (150-400ms) so the monitoring
    // cycle behaves like it would against a real API, instead of resolving
    // instantly and hiding timing bugs.
    await sleep(150 + Math.floor(Math.random() * 250));

    const key = `${offer.provider}:${offer.externalProductId}`;
    const entry = offersByKey.get(key);

    if (!entry) {
      return {
```

```
      provider: offer.provider,
      externalProductId: offer.externalProductId,
      success: false,
      errorMessage: `No fixture entry found for ${key}. Check fixtureData.json.`,
    };
  }

  // The deliberate-failure path: flip forceFail to true on any entry in
  // fixtureData.json to simulate a provider outage for that one offer only,
  // without affecting anything else (used by ACC-06, see Section 14 below).
  if (entry.forceFail) {
    return {
      provider: entry.provider,
      externalProductId: entry.externalProductId,
      success: false,
      errorMessage: 'Simulated provider failure (forceFail: true in fixtureData.jso
n).',
    };
  }

  return {
    provider: entry.provider,
    externalProductId: entry.externalProductId,
    title: entry.title,
    brand: entry.product.brand || null,
    model: entry.product.model || null,
    variant: entry.product.variant || null,
    priceInPaise: entry.priceInPaise,
    currency: entry.currency,
    availability: entry.availability,
    productUrl: entry.productUrl,
    imageUrl: entry.product.imageUrl || null,
    isSample: true, // always true for fixture data — see Document 1, Section 10.3/1
0.4
    success: true,
    errorMessage: null,
  };
}

module.exports = { fetchOffer };
```

6.3 Demoing a price drop live on Day 7

Document 1, Section 10.3 requires exactly this capability. To demo a price drop without waiting for a real price to change:

1. Open `server/src/providers/fixtureData.json`.
2. Pick an offer a demo alert is watching (e.g. the Redmi Note 13 5G Flipkart offer, currently `1699900` paise = ₹16,999).
3. Lower `priceInPaise` to a value below the alert's target (e.g. `1499900` = ₹14,999).
4. Save the file and restart the server (`npm run dev` in `server/`, or however Member 3's dev script is wired) so the new JSON is loaded.
5. Trigger a monitoring cycle (`POST /api/monitor/run` with the correct `X-Monitor-Secret` header, or the local dev script in Section 8.2) and watch the alert fire for real, end to end.

This is a deliberate, documented simplification for a 7-day demo (Document 1, Section 10.3) — it is not a claim that the price actually changed on Flipkart.

6.4 Testing the deliberate-failure path

`FLP-NOISE-PULSE2-BLK` has `forceFail: true` by default. Any attempt to fetch that specific offer returns `{ success: false, errorMessage: "Simulated provider failure..." }` every time, which is exactly what ACC-06 needs to verify: a forced failure must produce `Offer.lastStatus = "error"`, never a price of 0, and never a false alert (Section 10 below). To test a *different* offer's failure path, flip its `forceFail` to `true` instead and set this one back to `false`.

---

7. The Flipkart provider — built now, used only if/when approved (`server/src/providers/flipkart.js`)

Read this before the code: this file exists so the team isn't starting from zero the moment approval arrives, but as of writing this document, **no Flipkart credentials have been issued to this team.** Nothing in this file is called by the dispatcher by default (Section 5.3 routes `isSample: false, provider: "flipkart"` offers back to the fixture provider). The request/response shape below is inferred from Flipkart's public API documentation as carefully as possible without an approved account to test against — it has **not** been verified against a real, logged API response, and it may need small corrections once real credentials exist and the first real call is made.

7.1 Headers and request shape (from the official docs)

Flipkart's affiliate API docs specify two required headers once an account is approved:

- `Fk-Affiliate-Id` — the Affiliate Tracking ID issued after approval.
- `Fk-Affiliate-Token` — the API Token generated from the affiliate dashboard (API → API Token → Generate API Token).

7.2 Complete code

```
// server/src/providers/flipkart.js
  //
  // NOT WIRED INTO THE DISPATCHER BY DEFAULT. See providers/index.js and the
  // switch-over checklist in Section 7.4 of Document 4. This file becomes live
  // only after FLIPKART_AFFILIATE_ID / FLIPKART_AFFILIATE_TOKEN are real,
  // approved credentials AND a manual test call has been verified to return a
  // real price.
  //
  // The request/response shape here is inferred from Flipkart's official
  // affiliate API documentation (af_overview.html, af_prod_ref.html) and is
  // NOT yet verified against a live response — correct it against the real
  // API the moment credentials exist (Section 7.4, step 2).

  const axios = require('axios');

  const FLIPKART_API_BASE = 'https://affiliate-api.flipkart.net/affiliate/1.0';
  const REQUEST_TIMEOUT_MS = 8000;

  /**
   * @param {object} offer - { provider: "flipkart", externalProductId: string }
   * @returns {Promise<object>} a partial NormalizedResult (fetchedAt is added by
   *   the dispatcher, not here). Never throws.
   */
  async function fetchOffer(offer) {
    const affiliateId = process.env.FLIPKART_AFFILIATE_ID;
    const affiliateToken = process.env.FLIPKART_AFFILIATE_TOKEN;

    if (!affiliateId || !affiliateToken) {
      return {
        provider: 'flipkart',
        externalProductId: offer.externalProductId,
        success: false,
        errorMessage: 'FLIPKART_AFFILIATE_ID / FLIPKART_AFFILIATE_TOKEN are not set — F
  lipkart is not approved/configured yet.',
      };
    }

    try {
      const response = await axios.get(
        `${FLIPKART_API_BASE}/product.json`,
        {
```

```
        params: { id: offer.externalProductId },
        headers: {
          'Fk-Affiliate-Id': affiliateId,
          'Fk-Affiliate-Token': affiliateToken,
        },
        timeout: REQUEST_TIMEOUT_MS,
      }
    );

    const product = response.data && response.data.productBaseInfoV1;

    if (!product) {
      return {
        provider: 'flipkart',
        externalProductId: offer.externalProductId,
        success: false,
        errorMessage: 'Flipkart API response did not include productBaseInfoV1 — resp
onse shape may have changed; verify against the live docs.',
      };
    }

    const priceRupees = product.flipkartSpecialPrice?.amount ?? product.maximumRetail
Price?.amount;

    if (typeof priceRupees !== 'number') {
      return {
        provider: 'flipkart',
        externalProductId: offer.externalProductId,
        success: false,
        errorMessage: 'Flipkart API response did not include a usable price field.',
      };
    }

    return {
      provider: 'flipkart',
      externalProductId: offer.externalProductId,
      title: product.title || null,
      brand: product.attributes?.brand || null,
      model: product.attributes?.model || null,
      variant: product.attributes?.variant || null,
      priceInPaise: Math.round(priceRupees * 100), // Document 1 Section 5 precision
policy
      currency: 'INR',
```

```
      availability: product.inStock ? 'in_stock' : 'out_of_stock',
      productUrl: product.productUrl || null,
      imageUrl: Array.isArray(product.imageUrls?.['200x200']) ? product.imageUrls['20
0x200'][0] : null,
      isSample: false, // real data only — this is the whole point of this file
      success: true,
      errorMessage: null,
    };
  } catch (err) {
    // Never throw. Every failure mode — timeout, 4xx, 5xx, malformed JSON,
    // DNS failure — collapses into the same shape priceMonitor.js expects.
    const status = err.response ? err.response.status : null;
    const detail = err.response?.data?.message || err.message || 'Unknown Flipkart AP
I error';
    return {
      provider: 'flipkart',
      externalProductId: offer.externalProductId,
      success: false,
      errorMessage: status ? `Flipkart API error ${status}: ${detail}` : `Flipkart AP
I request failed: ${detail}`,
    };
  }
}

module.exports = { fetchOffer };
```

7.3 Why `isSample: false` only applies once this is actually wired in

Even after this file exists and is syntactically correct, **it does nothing** until `providers/index.js` is updated to call it instead of the fixture provider. That's intentional — it stops a half-tested integration from silently going live.

7.4 Switch-over checklist — follow this in order, do not skip steps

1. Confirm `FLIPKART_AFFILIATE_ID` and `FLIPKART_AFFILIATE_TOKEN` are real, approved credentials in `server/.env` (not placeholders).
2. Pick one real Flipkart product and manually test `flipkart.js` in isolation (e.g. a scratch script or a Jest test that calls `fetchOffer({ provider: 'flipkart', externalProductId: '<a real Flipkart product id>' })` directly) and confirm a real price comes back with `success: true`. **If the response shape doesn't match Section 7.2's assumptions, fix `flipkart.js` here, before touching the dispatcher.**

3. Only once step 2 is confirmed working, edit `providers/index.js`'s `isSample === false && provider === 'flipkart'` branch to call `flipkartProvider.fetchOffer(offer)` instead of `fixtureProvider.fetchOffer(offer)`.
4. In the database, flip `isSample` to `false` only for the specific Offer documents you have actually confirmed return a working real price — never flip it for every Flipkart-labeled offer at once based on one successful test.
5. Re-run the Jest test suite (Section 14) and the manual test checklist against the now-live offers before telling the rest of the team Flipkart is "live."
6. Update `docs/day1-retailer-findings.md` with the real approval date and the date real data went live.

---

8. GitHub Actions scheduler

8.1 Why this isn't an in-process scheduler

Document 1, Section 12 verified that **no free hosting tier reliably runs an always-on background worker or a guaranteed hourly cron** — specifically, Render's free Web Service tier sleeps after 15 minutes without inbound traffic, and Render has no free tier for a dedicated always-on Background Worker. If PriceRadar used something like `node-cron` running inside the Express app itself, that in-process scheduler would simply **stop firing the moment the free web service goes to sleep** — there's no code running at all during sleep, so a timer that depends on the process being alive is not a scheduler, it's a scheduler-shaped bug waiting to happen on exactly the free tier this project deploys to.

The decision (Document 1, Section 12): **GitHub Actions** calls `POST /api/monitor/run` on an hourly `cron:` schedule. This solves two problems with one free mechanism — it's the thing that actually decides "it's been an hour, go check prices," *and* because it's a real HTTPS request from the outside, it also wakes the sleeping Render instance as a side effect, so the very call that triggers monitoring is also what gets the server running again.

8.2 `.github/workflows/monitor.yml`

This file is configured together with Member 1/Document 5, since `BACKEND_URL` depends on where the backend is actually deployed. Here is the complete workflow:

```
name: Hourly Price Monitor

  on:
    schedule:
      - cron: '0 * * * *'   # every hour, on the hour (UTC) — GitHub Actions cron is "b
  est effort" and can be delayed a few minutes under load (Document 1, Section 12/18)
    workflow_dispatch:        # adds a manual "Run workflow" button in the Actions tab
  — useful on demo day to trigger a cycle on demand

  jobs:
    trigger-monitor:
      runs-on: ubuntu-latest
      steps:
        - name: Call POST /api/monitor/run
          run: |
            STATUS=$(curl -s -o response.json -w "%{http_code}" -X POST "${{ secrets.BA
  CKEND_URL }}/api/monitor/run" -H "X-Monitor-Secret: ${{ secrets.MONITOR_TRIGGER_SECRE
  T }}")
            echo "HTTP status: $STATUS"
            echo "Response body:"
            cat response.json
            if [ "$STATUS" -lt 200 ] || [ "$STATUS" -ge 300 ]; then
              echo "::error::Monitor trigger failed with HTTP status $STATUS"
              exit 1
            fi
```

This step does three things: makes the POST request with the required `X-Monitor-Secret` header (never a user JWT — Document 1, Section 6.6), prints the HTTP status and response body to the Action's log (so a failure is visible without guessing), and explicitly fails the whole workflow run (`exit 1`) if the status code isn't 2xx, so a broken monitoring call shows up as a red X in the Actions tab instead of silently doing nothing.

8.3 GitHub Actions secrets — what they are and where to set them

A **repository secret** is a value GitHub stores encrypted and injects into a workflow run as an environment-style variable, without ever showing it in logs or to anyone browsing the repo. Set these at **repo Settings → Secrets and variables → Actions → New repository secret**:

Secret name	Value
`BACKEND_URL`	The deployed backend's base URL, e.g. `https://priceradar-api.onrender.com` (set once Document 5's deployment is live — coordinate with Member 1)
`MONITOR_TRIGGER_SECRET`	The exact same value set in the deployed backend's `MONITOR_TRIGGER_SECRET` environment variable (Section 4.4) — these must match exactly or every run fails with `401 AUTH_INVALID`

Because `BACKEND_URL` doesn't exist until Document 5's deployment work lands, this workflow can be committed and merged early (it'll simply fail until the secrets are set, which is expected and fine on Day 1–5), but it shouldn't be expected to succeed until Day 6–7 per Document 1's milestone table (Section 15).

8.4 Local-dev-only script: `server/src/jobs/runMonitorLocally.js`

During development, you don't want to wait for an actual hourly cron (or even press the GitHub Actions manual-dispatch button) just to test a change. This plain Node script does the same HTTP call GitHub Actions does, but run by hand from PowerShell:

```
// server/src/jobs/runMonitorLocally.js
  //
  // LOCAL DEVELOPMENT CONVENIENCE ONLY (Document 1, Section 4/14). This is not
  // a cron library and nothing schedules it automatically — you run it by hand
  // whenever you want to trigger one monitoring cycle without waiting for the
  // real GitHub Actions schedule. Production scheduling is ALWAYS GitHub
  // Actions (Section 8 above), never this file, never node-cron inside the app.

  require('dotenv').config();
  const axios = require('axios');

  const PORT = process.env.PORT || 5000;
  const BASE_URL = process.env.LOCAL_MONITOR_URL || `http://localhost:${PORT}`;
  const SECRET = process.env.MONITOR_TRIGGER_SECRET;

  async function run() {
    if (!SECRET) {
      console.error('MONITOR_TRIGGER_SECRET is not set in server/.env — cannot call /ap
  i/monitor/run.');
      process.exit(1);
    }

    console.log(`Triggering a monitoring cycle against ${BASE_URL}/api/monitor/run ...
  `);

    try {
      const response = await axios.post(
        `${BASE_URL}/api/monitor/run`,
        {},
        { headers: { 'X-Monitor-Secret': SECRET }, timeout: 30000 }
      );
      console.log('Status:', response.status);
      console.log('Response:', JSON.stringify(response.data, null, 2));
    } catch (err) {
      if (err.response) {
        console.error('Monitor call failed. Status:', err.response.status);
        console.error('Response body:', JSON.stringify(err.response.data, null, 2));
      } else {
        console.error('Monitor call failed:', err.message);
      }
      process.exit(1);
    }
```

```
}

run();
```

Run it with:

```
cd D:\Projects\PriceRadar\server
  node src/jobs/runMonitorLocally.js
```

This requires your local server (`npm run dev`) to already be running in another PowerShell window, and requires Member 1's `POST /api/monitor/run` route to exist (Section 9 explains what to do before that route exists).

---

9. `POST /api/monitor/run` and `priceMonitor.js` — interface Member 4 must honor, not implement

`server/src/services/priceMonitor.js` and the `routes/monitor.js` route are owned by Member 1 (Document 1, Section 14). Do not create or edit either file without coordinating first. This section exists so you know exactly what that file will call into, and so you can build and test the provider layer (Sections 5–7) independently, before Member 1's code even exists.

9.1 What `priceMonitor.js` will do, by contract

Per Document 1, Sections 2 and 6.6: when `POST /api/monitor/run` is called with a valid `X-Monitor-Secret` header, `priceMonitor.js` will:

1. Load every **active, distinct** `Offer` that's on at least one watchlist or alert (one offer fetched once per cycle, no matter how many users track it — Document 1, Section 2).
2. For each offer, call `providers/index.js`'s `fetchOffer(offer)` — **this is the only function of yours it calls directly.**
3. Write a `PriceObservation` document from the result (Document 1, Section 5.4: `{ offerId, priceInPaise (only on success), status: "success"|"failure", errorMessage (only on failure), fetchedAt }`).
4. Update the matching `Offer` document's `lastFetchedAt`, `lastSuccessAt`, `lastStatus`, `lastError`, and (only on success) `priceInPaise` (Document 1, Section 5.3 — see Section 10 below for the exact pattern).
5. Call `alertEvaluator.js` for every alert tied to that offer.
6. Respond `202 { started: true, offersChecked: n, triggeredAlerts: n }`.

9.2 Testing your provider layer before that route exists

Because your contract with `priceMonitor.js` is "it calls `fetchOffer(offer)` and gets back a `NormalizedResult`," you can fully verify your half of that contract with a plain Jest test that never needs Member 1's code, a running Express server, or a database:

```
// server/tests/providers.dispatcher.test.js
  const { fetchOffer } = require('../src/providers');

  describe('providers/index.js dispatcher', () => {
    test('fixture offer returns a complete, successful NormalizedResult', async () => {
      const offer = { provider: 'flipkart', externalProductId: 'FLP-SGM14-128-SLV', isS
  ample: true };
      const result = await fetchOffer(offer);

      expect(result.success).toBe(true);
      expect(typeof result.priceInPaise).toBe('number');
      expect(result.isSample).toBe(true);
      expect(result.currency).toBe('INR');
      expect(['in_stock', 'out_of_stock', 'unknown']).toContain(result.availability);
      expect(typeof result.fetchedAt).toBe('string');
    });

    test('the deliberate-failure fixture entry returns success:false, not a throw', asy
  nc () => {
      const offer = { provider: 'flipkart', externalProductId: 'FLP-NOISE-PULSE2-BLK',
  isSample: true };
      const result = await fetchOffer(offer);

      expect(result.success).toBe(false);
      expect(typeof result.errorMessage).toBe('string');
      expect(result.errorMessage.length).toBeGreaterThan(0);
    });

    test('a non-sample Amazon offer never attempts a live call', async () => {
      const offer = { provider: 'amazon', externalProductId: 'AMZ-ANYTHING', isSample:
  false };
      const result = await fetchOffer(offer);

      expect(result.success).toBe(false);
      expect(result.errorMessage).toMatch(/no live provider/i);
    });

    test('an unknown fixture key fails cleanly instead of throwing', async () => {
      const offer = { provider: 'flipkart', externalProductId: 'DOES-NOT-EXIST', isSamp
  le: true };
      await expect(fetchOffer(offer)).resolves.toMatchObject({ success: false });
```

```
    });
  });
```

This test file is independent of the real `priceMonitor.js` and can be written and passing well before Member 1's monitoring orchestration exists — it's testing exactly the boundary you own.

---

10. Failed-fetch and stale-data handling

This is the single most important correctness rule in the whole document — Document 1's non-functional requirement "Resilience" (Section 1.3) exists specifically so a broken provider call never becomes a false price or a false alert.

10.1 The rule, precisely

- A failed fetch **never** sets `Offer.priceInPaise` to `0`, and never overwrites the last known good price at all.
- `Offer.lastFetchedAt` is updated on **every** attempt — success or failure — because it answers "when did we last try," not "when did we last succeed."
- `Offer.lastSuccessAt` is updated **only** on success — it answers "when did we last actually get a real price."
- `Offer.lastStatus` becomes `"error"` (with `Offer.lastError` set to a human-readable reason) on failure, `"ok"` on success. It's `"stale"` when neither has happened recently enough to trust — specifically: **no successful fetch has ever happened yet**, or **`lastSuccessAt` is older than a reasonable staleness window** (a sensible default for an hourly-cycle system: if it's been more than ~3 hours since the last success, something is wrong even if the most recent individual attempt technically "succeeded" in writing a status — in practice, for this project, `lastStatus` is set directly to `"ok"` / `"error"` by each monitoring attempt, and a separate read-time check in the UI/API layer treats an offer as effectively stale when `lastSuccessAt` is older than the staleness window, regardless of what `lastStatus` currently says).
- `Offer.lastError` is **cleared** (`null`) the next time a fetch succeeds — it should only ever reflect the *most recent* failure, never accumulate old ones.

10.2 The exact update pattern

This exact logic lives in `priceMonitor.js` (Member 1's file), but you need to understand it precisely because a bug here would make your provider's `success: false` meaningless. This is the pattern Member 1's code follows for every offer, once per cycle:

```
// Illustrative only — the real implementation lives in priceMonitor.js (Member 1).
  // Shown here so Member 4 can verify the provider contract is sufficient to
  // support this logic correctly, and to catch a mismatch early.

  const result = await providers.fetchOffer(offer); // Member 4's dispatcher

  const update = {
    lastFetchedAt: result.fetchedAt, // ALWAYS updated, success or failure
  };

  if (result.success) {
    update.priceInPaise = result.priceInPaise; // ONLY overwritten on success
    update.lastSuccessAt = result.fetchedAt;    // ONLY updated on success
    update.lastStatus = 'ok';
    update.lastError = null;                    // cleared on success
  } else {
    update.lastStatus = 'error';
    update.lastError = result.errorMessage;
    // priceInPaise and lastSuccessAt are deliberately left untouched here —
    // the last known good price and timestamp survive a failed attempt.
  }

  await Offer.findByIdAndUpdate(offer._id, update);

  await PriceObservation.create({
    offerId: offer._id,
    ...(result.success
      ? { priceInPaise: result.priceInPaise, status: 'success' }
      : { status: 'failure', errorMessage: result.errorMessage }),
    fetchedAt: result.fetchedAt,
  });
```

The reason this is safe: because your provider functions (Sections 6 and 7) **never throw** and always return a well-formed `{ success, errorMessage }` pair, this `if (result.success)` branch is the *only* place that decision needs to be made, and it's made exactly once, consistently, for every provider.

10.3 Why this matters for alert logic

Document 1, Section 9.3's last line: "A failed `PriceObservation` (`status: "failure"`) never changes `armed` and never triggers an alert." That rule only works if a failure is reliably recorded as `status: "failure"` with no `priceInPaise` field at all — if a provider ever returned `priceInPaise: 0` on failure instead of `success: false`, `alertEvaluator.js` would see a real (if wrong) number and could trigger a false target-price alert ("price dropped to ₹0!"). This is exactly what ACC-06 tests (Section 14).

11. Brevo email setup and implementation

Task NOTIF-01: Brevo transactional email service

A. Task ID and title

`NOTIF-01` — Set up Brevo and implement `server/src/services/notifications/email.js`.

B. Owner and reviewer

Owner: Member 4. Reviewer: Member 1 (per Document 1, Section 14 — Member 1 reviews changes touching shared/notification-sensitive code).

C. Purpose

Deliver a real email to a user when one of their alerts triggers, using Brevo's free transactional email API, and never crash the monitoring cycle if that email fails to send.

D. Prerequisites

- A Day-1/Day-2 setup step (Document 1, Section 11) — do this early, sender verification can take a little time.
- A team email address the team controls (for sender verification).

E. Exact files

- Create: `server/src/services/notifications/email.js`.
- Edit: `server/.env` and `server/.env.example` (add `BREVO_API_KEY`, `EMAIL_FROM` — likely already added in Section 4.4).
- Do not edit: `server/src/services/alertEvaluator.js` (Member 1's file, which will call `sendEmail()`).

F. Commands

```
cd D:\Projects\PriceRadar\server
  # axios is already installed (Section 4.2) — no new package needed for raw API calls.
```

Why raw Axios instead of Brevo's own SDK (`@getbrevo/brevo` / `sib-api-v3-sdk`)? Both work. Raw Axios is simpler for a beginner team: it's one HTTP POST with a JSON body and one header, uses the same HTTP client already used everywhere else in the project (Document 1, Section 3), and avoids learning a second SDK's object model on top of the one request this project actually needs. The SDK adds real value at a larger scale (typed clients, more endpoints) that this project doesn't need.

G. Implementation

1. **Create the Brevo account.** Go to **https://www.brevo.com**, sign up for a free account. Document 1, Section 11 verified: 300 emails/day, no time limit, up to 100,000 stored contacts, **1 user per free account**. Use **one shared team account** — do not create four separate Brevo accounts, since each has its own daily cap and would fragment delivery history across accounts for no benefit.
2. **Verify a sender.** In the Brevo dashboard, go to **Senders, Domains & Dedicated IPs → Senders → Add a sender**. For a 7-day training project, you almost certainly don't have a custom domain — that's fine. Use a real inbox the team controls (a personal or team Gmail-type address works) as the verified sender. Brevo sends a verification email to that inbox; click the confirmation link. **This is a documented simplification for a demo project, not a production pattern** — in production you'd normally verify a domain you own, not a personal inbox, but domain verification needs DNS access this team doesn't have for a 7-day sprint.
3. **Get the API key.** Dashboard → **Settings → SMTP & API → API Keys → Generate a new API key**. Copy it immediately (Brevo only shows it once).
4. Fill in `server/.env`:

```
BREVO_API_KEY=xkeysib-...your real key...
  EMAIL_FROM=your-verified-sender@example.com
```

5. **What "transactional email API" and "sender verification" mean, briefly:** A *transactional* email is a one-to-one, triggered message (a password reset, an order confirmation, a price alert) as opposed to a bulk marketing blast — Brevo's transactional API is built for exactly this pattern and is what the free tier's 300/day limit applies to. *Sender verification* exists because email providers (Gmail, Outlook) aggressively spam-filter mail claiming to come from an address the sending service hasn't proven it's allowed to send on behalf of — without it, your alert emails would likely never reach an inbox at all, or would land in spam.
6. Create `server/src/services/notifications/email.js`:

```
// server/src/services/notifications/email.js
  //
  // Sends a transactional email via Brevo's API. Never throws — every failure
  // mode (bad API key, unverified sender, network error, rate limit) resolves
  // to a result object the caller can log into a NotificationEvent
  // (Document 1, Section 5.7) instead of crashing the monitoring cycle.

  const axios = require('axios');

  const BREVO_SEND_URL = 'https://api.brevo.com/v3/smtp/email';
  const REQUEST_TIMEOUT_MS = 10000;

  /**
   * @param {object} params
   * @param {string} params.to - recipient email address
   * @param {string} params.subject
   * @param {string} params.text - plain-text body
   * @param {string} [params.html] - optional HTML body; falls back to a simple
   *   wrap of `text` if omitted
   * @returns {Promise<{success: boolean, providerMessageId?: string, errorMessage?: st
  ring}>}
   */
  async function sendEmail({ to, subject, text, html }) {
    const apiKey = process.env.BREVO_API_KEY;
    const fromAddress = process.env.EMAIL_FROM;

    if (!apiKey || !fromAddress) {
      return {
        success: false,
        errorMessage: 'BREVO_API_KEY or EMAIL_FROM is not configured in server/.env.',
      };
    }

    try {
      const response = await axios.post(
        BREVO_SEND_URL,
        {
          sender: { email: fromAddress, name: 'PriceRadar Alerts' },
          to: [{ email: to }],
          subject,
          textContent: text,
          htmlContent: html || `<p>${text}</p>`,
```

```
      },
      {
        headers: {
          'api-key': apiKey,
          'Content-Type': 'application/json',
          Accept: 'application/json',
        },
        timeout: REQUEST_TIMEOUT_MS,
      }
    );

    return {
      success: true,
      providerMessageId: response.data && response.data.messageId ? response.data.mes
sageId : null,
    };
  } catch (err) {
    // Never throw — see module header. Distinguish 401 (bad key) from 400
    // (e.g. unverified sender) where possible, since they need different fixes
    // (Section 15, Debugging).
    const status = err.response ? err.response.status : null;
    const body = err.response ? err.response.data : null;
    const detail = (body && (body.message || body.code)) || err.message || 'Unknown e
mail error';
    return {
      success: false,
      errorMessage: status ? `Brevo API error ${status}: ${detail}` : `Brevo request
failed: ${detail}`,
    };
  }
}

module.exports = { sendEmail };
```

H. Beginner explanation

`sendEmail` is a thin wrapper around one HTTPS POST. The important design choice is the same one as the provider functions in Sections 6–7: **it never throws**, it always resolves to `{ success, ... }`. That's what lets the code that calls it (`alertEvaluator.js`, via the dedup/retry logic in Section 13) treat "email sent" and "email failed" as two outcomes to record, not as a crash to catch.

I. Environment configuration

`BREVO_API_KEY` (secret), `EMAIL_FROM` (not secret, but must be a verified sender) — both in `server/.env`, never in frontend code, never committed.

J. Testing

Automated (mock the HTTP call — see Section 14). Manual: write a tiny one-off script or use the Jest test to call `sendEmail({ to: 'your-own-test-inbox@example.com', subject: 'PriceRadar test', text: 'If you see this, Brevo is wired up correctly.' })` and confirm the email actually lands.

K. Expected results

`{ success: true, providerMessageId: '<...>' }` and a real email in the test inbox within a few seconds. A bad API key: `{ success: false, errorMessage: 'Brevo API error 401: ...' }`.

L. Troubleshooting

See Section 15.

M. Acceptance criteria

- `sendEmail()` never throws under any tested failure condition.
- A real email, sent to a real inbox the team controls, is received and readable.
- `BREVO_API_KEY` does not appear anywhere in committed code or logs.

N. Git workflow

See Section 16 (`feature/email-notifications`).

O. Completion checklist

- [ ] Shared Brevo account created, sender verified.
- [ ] `BREVO_API_KEY` / `EMAIL_FROM` set in local `.env`, placeholders added to `.env.example`.
- [ ] `email.js` committed, Jest test passing, one real manual send confirmed.

P. AI-assisted debugging prompt

> *"Here is `server/src/services/notifications/email.js` and the exact error I'm seeing from Brevo: [paste]. Inspect the code and the error response body, identify the root cause (do not guess between a 401 auth problem and a 400 sender-verification problem without checking the status code), suggest the smallest safe change, and tell me how to verify the fix with a real test send."*

---

12. Telegram bot setup and implementation

Task NOTIF-02: Telegram bot and account linking

A. Task ID and title

`NOTIF-02` — Create the Telegram bot, implement `server/src/services/notifications/telegram.js`, and build the account-linking flow.

B. Owner and reviewer

Owner: Member 4. Reviewer: Member 1. **Note:** the linking flow touches `User.telegramChatId` (Document 1, Section 5.1, owned at the model level by Member 3) and proposes one new endpoint not yet in Document 1's contract (Section 12.3 below) — both points need a short sync with Member 1/Member 3 before merging.

C. Purpose

Let a user link their Telegram account to PriceRadar, and deliver a real Telegram message when one of their alerts triggers.

D. Prerequisites

A Telegram account to create the bot with.

E. Exact files

- Create: `server/src/services/notifications/telegram.js`, `server/src/services/notifications/telegramLinkCodes.js`.
- Propose to Member 3 (do not edit directly): a `POST /api/telegram/link-code` route/controller, and the implementation detail of the existing `POST /api/telegram/link` controller (both live under `server/src/routes/` and `server/src/controllers/`, which are Member 3's files per Document 1, Section 14).

F. Commands

```
cd D:\Projects\PriceRadar\server
  # axios already installed.
```

G. Implementation

Step 1 — create the bot. Open Telegram, search for **@BotFather**, start a chat, send `/newbot`. Follow the prompts: choose a display name (e.g. "PriceRadar Alerts"), then a username ending in `bot` (e.g. `PriceRadarAlertsBot`). BotFather replies with a token like `123456789:AAH...`. **Treat this token as a secret immediately** — it's the only thing needed to send messages as your bot.

```
# server/.env
  TELEGRAM_BOT_TOKEN=123456789:AAH...your-real-token...
```

Step 2 — understand the linking flow this project uses. Document 1, Section 6.5 specifies: *"the user starts the bot and sends `/start <linkCode>` ... The bot backend records the mapping; the frontend polls or the user pastes the code back — see Document 4 for the exact mechanism."* Document 1 defines `POST /api/telegram/link` (`{linkCode}` → `{linked: true}`) but does **not** specify how a `linkCode` is generated in the first place. Here is the concrete mechanism this document adds:

> ***Addition to Document 1's contract — flag this to Member 1 for the master doc:*** *this document proposes `POST /api/telegram/link-code` (auth required, no body, returns `{ linkCode, botUsername, expiresAt }`) as the endpoint that generates a short-lived code tied to `req.user.id`. Document 1 Section 6.5 does not specify this endpoint; it should be added there once agreed.*

The full flow:

1. User, logged in, opens Notification Preferences and clicks "Link Telegram." Frontend calls `POST /api/telegram/link-code`.
2. Backend generates a short random code (e.g. 6 characters), stores `{ code → { userId, expiresAt } }`, returns it along with the bot's username.
3. Frontend shows: *"Open Telegram, message @PriceRadarAlertsBot, and send: `/start <code>`"*.
4. The user does that in Telegram. A **long-polling listener** (this project's choice — see below) running inside the backend process receives the update, matches `<code>` to a pending entry, and records the `chatId` Telegram gave it alongside that code.
5. The frontend (via polling `GET /api/notification-preferences`, or a "I've sent it — confirm" button) calls `POST /api/telegram/link { linkCode }`, authenticated as the same user who generated the code. The backend looks up the code, confirms it belongs to `req.user.id` **and** that step 4 already recorded a `chatId` for it, sets `User.telegramChatId`, deletes the code, and returns `{ linked: true }`.

The code-ownership check in step 5 matters: it stops a code meant for one user from being claimed by a different logged-in session, even though in practice a code is only ever shown to the user who generated it.

Step 3 — why long polling, not a webhook, for this project. Telegram supports two ways to receive bot updates: a **webhook** (Telegram pushes updates to a public HTTPS URL you register) or **long polling** (your server repeatedly asks Telegram's `getUpdates` endpoint "anything new?"). A webhook is more "production-correct" once the backend is deployed and has a stable public URL, but it adds real complexity for a beginner team during local development — you'd need a public HTTPS tunnel (like ngrok) just to test linking on your own laptop, since Telegram can't push updates to `localhost`. Long polling needs no public URL at all, which makes it dramatically simpler to run and debug locally on Day 1–5. **Recommendation: use long polling for the whole 7-day project.** Switching to a webhook after deployment is a reasonable stretch goal, not required for the MVP — note it as a possible Day 6/7 improvement if time allows, never as a blocker.

Step 4 — the shared in-memory code store.

```
// server/src/services/notifications/telegramLinkCodes.js
  //
  // Short-lived Telegram account-linking codes, shared between the
  // POST /api/telegram/link-code and POST /api/telegram/link routes (Member 3's
  // files) and the long-polling listener below (Member 4's file). In-memory by
  // design for a 7-day MVP: a code lives for a few minutes, which is shorter
  // than a typical gap between generating a code and using it, so losing
  // pending codes on a server restart/sleep (Document 1, Section 12 — Render's
  // free tier sleeps and can restart) is an acceptable, documented trade-off,
  // not a production-grade store.

  const CODE_TTL_MS = 10 * 60 * 1000; // 10 minutes
  const codes = new Map(); // code -> { userId, chatId: string|null, expiresAt: number
  }

  function generateCode(userId) {
    const code = Math.random().toString(36).slice(2, 8).toUpperCase(); // e.g. "K3F9QA"
    codes.set(code, { userId, chatId: null, expiresAt: Date.now() + CODE_TTL_MS });
    return { code, expiresAt: new Date(Date.now() + CODE_TTL_MS).toISOString() };
  }

  function recordChatIdForCode(code, chatId) {
    const entry = codes.get(code);
    if (!entry || entry.expiresAt < Date.now()) return false;
    entry.chatId = chatId;
    return true;
  }

  function consumeCode(code, userId) {
    const entry = codes.get(code);
    if (!entry || entry.expiresAt < Date.now()) return null;
    if (entry.userId !== userId) return null; // code belongs to a different user
    if (!entry.chatId) return null; // user hasn't completed the Telegram side yet
    codes.delete(code);
    return entry.chatId;
  }

  module.exports = { generateCode, recordChatIdForCode, consumeCode };
```

Step 5 — `server/src/services/notifications/telegram.js` (the `sendMessage` function plus the long-polling listener):

```
// server/src/services/notifications/telegram.js
  //
  // sendMessage() delivers a real Telegram message via the Bot API and never
  // throws. startLinkListener() is the minimal long-polling loop that drives
  // account linking (Section 12 above) — it is NOT a general chatbot framework,
  // it only understands "/start <code>".

  const axios = require('axios');
  const { recordChatIdForCode } = require('./telegramLinkCodes');

  function apiBase() {
    return `https://api.telegram.org/bot${process.env.TELEGRAM_BOT_TOKEN}`;
  }

  const REQUEST_TIMEOUT_MS = 10000;

  /**
   * @param {object} params
   * @param {string} params.chatId
   * @param {string} params.text
   * @returns {Promise<{success: boolean, messageId?: number, errorMessage?: string}>}
   */
  async function sendMessage({ chatId, text }) {
    if (!process.env.TELEGRAM_BOT_TOKEN) {
      return { success: false, errorMessage: 'TELEGRAM_BOT_TOKEN is not configured in s
  erver/.env.' };
    }
    if (!chatId) {
      return { success: false, errorMessage: 'No telegramChatId on file for this user —
  they have not linked Telegram yet.' };
    }

    try {
      const response = await axios.post(
        `${apiBase()}/sendMessage`,
        { chat_id: chatId, text, parse_mode: 'HTML' },
        { timeout: REQUEST_TIMEOUT_MS }
      );
      return { success: true, messageId: response.data.result.message_id };
    } catch (err) {
      const status = err.response ? err.response.status : null;
      const detail = err.response?.data?.description || err.message || 'Unknown Telegra
```

```
m error';
    return {
      success: false,
      errorMessage: status ? `Telegram API error ${status}: ${detail}` : `Telegram re
quest failed: ${detail}`,
    };
  }
}

// ---- Minimal long-polling listener for the linking flow only ----

let pollingOffset = 0;
let pollingTimer = null;

async function pollOnce() {
  try {
    const response = await axios.get(`${apiBase()}/getUpdates`, {
      params: { offset: pollingOffset, timeout: 20 },
      timeout: 25000,
    });

    for (const update of response.data.result || []) {
      pollingOffset = update.update_id + 1;

      const message = update.message;
      const text = message && message.text;
      const match = text && text.match(/^\/start\s+(\S+)$/);
      if (match) {
        const code = match[1].toUpperCase();
        const chatId = String(message.chat.id);
        const recorded = recordChatIdForCode(code, chatId);
        await sendMessage({
          chatId,
          text: recorded
            ? 'Code received! Go back to PriceRadar and confirm linking on the Notifi
cation Preferences page.'
            : 'That code was not recognized or has expired — generate a new one from
PriceRadar and try again.',
        });
      }
    }
  } catch (err) {
    console.error('[telegram] polling error:', err.message);
```

```
  }
}

function startLinkListener() {
  if (!process.env.TELEGRAM_BOT_TOKEN) {
    console.warn('[telegram] TELEGRAM_BOT_TOKEN not set — link listener not starte
d.');
    return;
  }
  if (pollingTimer) return; // already running
  console.log('[telegram] starting long-polling link listener...');
  pollingTimer = setInterval(pollOnce, 2000);
}

function stopLinkListener() {
  if (pollingTimer) {
    clearInterval(pollingTimer);
    pollingTimer = null;
  }
}

module.exports = { sendMessage, startLinkListener, stopLinkListener };
```

Step 6 — start the listener. `startLinkListener()` needs to run once when the server starts — that means one line added to `server/src/server.js`, which is Member 3's file. **Add this in coordination with Member 3, as a one-line addition, not a rewrite:**

```
// In server/src/server.js, after the server starts listening:
  const { startLinkListener } = require('./services/notifications/telegram');
  startLinkListener();
```

H. Beginner explanation

Long polling works by repeatedly asking Telegram "anything new since update `X`?" every couple of seconds. Each response includes an `update_id`; by always asking for updates *after* the last one you've seen (`offset`), Telegram never re-sends something you've already processed. This project's listener only cares about one message shape — `/start <code>` — everything else is ignored, which keeps it genuinely minimal rather than a general-purpose bot framework.

I. Environment configuration

`TELEGRAM_BOT_TOKEN` (secret) in `server/.env`.

J. Testing

Manual: generate a code (once the `/api/telegram/link-code` route exists), send `/start <code>` to the real bot from a real Telegram account, confirm the bot replies, confirm `POST /api/telegram/link` sets `telegramChatId`, then confirm `sendMessage` actually delivers a message to that chat.

K. Expected results

A real message appears in the linked Telegram chat within a few seconds of calling `sendMessage`.

L. Troubleshooting

See Section 15 ("chat not found").

M. Acceptance criteria

- `sendMessage()` never throws.
- `/start <code>` from a real account results in `User.telegramChatId` being set after the confirm step.
- A message sent via `sendMessage()` after linking is actually received in Telegram.

N. Git workflow

See Section 16 (`feature/telegram-notifications`).

O. Completion checklist

- [ ] Bot created via @BotFather, token in `.env`.
- [ ] `telegramLinkCodes.js` and `telegram.js` committed.
- [ ] `POST /api/telegram/link-code` proposal handed to Member 3/Member 1, flagged as a Document 1 addition.
- [ ] One real end-to-end link + message delivery confirmed manually.

P. AI-assisted debugging prompt

> *"Here is `server/src/services/notifications/telegram.js` and the error/response I'm getting from the Telegram Bot API: [paste]. Inspect the code and the response body, identify whether this is a bad token, an unlinked chat ('chat not found'), or a polling offset bug, suggest the smallest safe fix, and tell me how to verify it with a real message."*

---

13. Notification dedup and retry policy

`NotificationEvent` (Document 1, Section 5.7) is owned by Member 4 — Member 3 creates the Mongoose model file (ask Member 3 to add the unique compound index `{alertId, dedupeKey}` when the model is created), but the logic that decides *when* to write one, and what `status` to give it, lives in

Member 4's notification services, because it's the natural continuation of "did the send succeed" that `email.js` / `telegram.js` already answer.

13.1 `dedupeKey` — the exact format

Per Document 1, Section 9.3:

```
dedupeKey = `${alertId}:${offerId}:${priceInPaise}:${triggeredAtHour}`
```

This is a **second, independent safety net** on top of the `armed` boolean `alertEvaluator.js` manages — even if the evaluator somehow runs twice for the same hour (a manual `POST /api/monitor/run` plus the scheduled GitHub Actions run landing in the same hour, for example), the unique index on `{alertId, dedupeKey}` means the second attempt is recorded as `status: "skipped_duplicate"` instead of sending a second message.

13.2 The orchestrating function

This is what `alertEvaluator.js` (Member 1's file) calls, once per channel, when it decides an alert should fire. It lives in Member 4's area since it owns the NotificationEvent write:

```
// server/src/services/notifications/sendAndRecordNotification.js
  //
  // Called by alertEvaluator.js (Member 1) exactly once per channel per
  // triggered alert. Builds the dedupeKey (Document 1, Section 9.3), checks for
  // a duplicate, and on a genuinely new event, attempts delivery with one
  // bounded retry before recording the final outcome.

  const NotificationEvent = require('../../models/NotificationEvent'); // Member 3's mo
  del file
  const { sendEmail } = require('./email');
  const { sendMessage: sendTelegramMessage } = require('./telegram');

  const RETRY_DELAY_MS = 3000;

  function sleep(ms) {
    return new Promise((resolve) => setTimeout(resolve, ms));
  }

  /**
   * @param {object} params
   * @param {string} params.userId
   * @param {string} params.alertId
   * @param {string} params.offerId
   * @param {"email"|"telegram"} params.channel
   * @param {number} params.priceInPaise - the triggering price
   * @param {string} params.message - the text to send
   * @param {string} params.recipient - email address or telegramChatId, depending on c
  hannel
   * @returns {Promise<object>} the saved NotificationEvent
   */
  async function sendAndRecordNotification({ userId, alertId, offerId, channel, priceIn
  Paise, message, recipient }) {
    const triggeredAtHour = new Date().toISOString().slice(0, 13); // "2026-10-13T14" —
  one bucket per hour
    const dedupeKey = `${alertId}:${offerId}:${priceInPaise}:${triggeredAtHour}`;

    // Bounded retry: try once, and on a failed attempt, try exactly one more
    // time after a short delay. No infinite retry loop — this job already runs
    // every hour on its own schedule (GitHub Actions, Section 8), so a
    // persistent failure will simply be retried again next cycle rather than
    // this one call looping and holding up the whole monitoring run.
    async function attemptSend() {
```

```
    if (channel === 'email') {
      return sendEmail({ to: recipient, subject: 'PriceRadar price alert', text: mess
age });
    }
    if (channel === 'telegram') {
      return sendTelegramMessage({ chatId: recipient, text: message });
    }
    return { success: false, errorMessage: `Unknown channel "${channel}"` };
  }

  let outcome = await attemptSend();
  if (!outcome.success) {
    await sleep(RETRY_DELAY_MS);
    outcome = await attemptSend();
  }

  const eventData = {
    userId,
    alertId,
    channel,
    message,
    dedupeKey,
    status: outcome.success ? 'sent' : 'failed',
    errorMessage: outcome.success ? undefined : outcome.errorMessage,
  };

  try {
    return await NotificationEvent.create(eventData);
  } catch (err) {
    // Mongo duplicate-key error (code 11000) on the {alertId, dedupeKey}
    // unique index means another process already recorded this exact event —
    // record it as skipped rather than letting the error bubble up and crash
    // the monitoring cycle.
    if (err.code === 11000) {
      return NotificationEvent.findOne({ alertId, dedupeKey });
    }
    throw err; // a genuine, unexpected DB error — let it surface, don't hide it
  }
}

module.exports = { sendAndRecordNotification };
```

13.3 Why no infinite retry

A scheduled job that runs every hour already *is* a retry mechanism at the system level — if a notification fails this cycle, the exact same alert condition (if still true) will be evaluated again next cycle. An unbounded retry loop inside a single request would risk holding the whole `/api/monitor/run` call open indefinitely on a persistent outage (e.g. Brevo down for an hour), delaying every *other* offer's monitoring in the same cycle. One retry, a few seconds apart, catches a transient blip (a dropped connection, a momentary rate limit) without risking that.

---

14. Testing

14.1 Jest unit tests

```
// server/tests/providers.fixture.test.js
  const { fetchOffer } = require('../src/providers/fixture');

  describe('fixture provider', () => {
    test('returns a correctly shaped successful result', async () => {
      const result = await fetchOffer({ provider: 'amazon', externalProductId: 'AMZ-SGM
  14-128-SLV' });
      expect(result).toMatchObject({
        provider: 'amazon',
        externalProductId: 'AMZ-SGM14-128-SLV',
        currency: 'INR',
        isSample: true,
        success: true,
      });
      expect(typeof result.priceInPaise).toBe('number');
    });

    test('the forceFail entry returns success:false, never throws', async () => {
      await expect(
        fetchOffer({ provider: 'flipkart', externalProductId: 'FLP-NOISE-PULSE2-BLK' })
      ).resolves.toMatchObject({ success: false });
    });
  });
```

```
// server/tests/notifications.email.test.js
  jest.mock('axios');
  const axios = require('axios');
  const { sendEmail } = require('../src/services/notifications/email');

  describe('email service', () => {
    beforeEach(() => {
      process.env.BREVO_API_KEY = 'test-key';
      process.env.EMAIL_FROM = 'alerts@example.com';
    });

    test('returns success on a 201 from Brevo', async () => {
      axios.post.mockResolvedValue({ data: { messageId: 'abc123' } });
      const result = await sendEmail({ to: 'user@example.com', subject: 'Test', text:
  'hi' });
      expect(result).toEqual({ success: true, providerMessageId: 'abc123' });
    });

    test('returns success:false, not a throw, on a Brevo error', async () => {
      axios.post.mockRejectedValue({ response: { status: 401, data: { message: 'Invalid
  API key' } } });
      const result = await sendEmail({ to: 'user@example.com', subject: 'Test', text:
  'hi' });
      expect(result.success).toBe(false);
      expect(result.errorMessage).toMatch(/401/);
    });
  });
```

```
// server/tests/notifications.telegram.test.js
  jest.mock('axios');
  const axios = require('axios');
  const { sendMessage } = require('../src/services/notifications/telegram');

  describe('telegram service', () => {
    beforeEach(() => {
      process.env.TELEGRAM_BOT_TOKEN = 'test-token';
    });

    test('returns success on a valid send', async () => {
      axios.post.mockResolvedValue({ data: { result: { message_id: 42 } } });
      const result = await sendMessage({ chatId: '123456', text: 'Price dropped!' });
      expect(result).toEqual({ success: true, messageId: 42 });
    });

    test('returns success:false on "chat not found"', async () => {
      axios.post.mockRejectedValue({ response: { status: 400, data: { description: 'Bad
  Request: chat not found' } } });
      const result = await sendMessage({ chatId: '000000', text: 'Price dropped!' });
      expect(result.success).toBe(false);
      expect(result.errorMessage).toMatch(/chat not found/);
    });
  });
```

```
// server/tests/notifications.dedup.test.js
  const mongoose = require('mongoose');
  const { MongoMemoryServer } = require('mongodb-memory-server'); // if the team adds t
  his dev dependency; otherwise run against a disposable test DB
  const NotificationEvent = require('../src/models/NotificationEvent');
  const { sendAndRecordNotification } = require('../src/services/notifications/sendAndR
  ecordNotification');

  jest.mock('../src/services/notifications/email', () => ({
    sendEmail: jest.fn().mockResolvedValue({ success: true, providerMessageId: 'x' }),
  }));

  describe('notification dedup', () => {
    const params = {
      userId: new mongoose.Types.ObjectId().toString(),
      alertId: new mongoose.Types.ObjectId().toString(),
      offerId: new mongoose.Types.ObjectId().toString(),
      channel: 'email',
      priceInPaise: 1499900,
      message: 'Price dropped to ₹14,999',
      recipient: 'user@example.com',
    };

    test('a second identical call in the same hour is skipped, not re-sent', async () =
  > {
      const first = await sendAndRecordNotification(params);
      expect(first.status).toBe('sent');

      const second = await sendAndRecordNotification(params);
      expect(second.dedupeKey).toBe(first.dedupeKey);
      // The second call should resolve to the SAME stored event, not a new "sent" one.
      const count = await NotificationEvent.countDocuments({ dedupeKey: first.dedupeKey
  });
      expect(count).toBe(1);
    });
  });
```

(The dedup test needs a real or in-memory MongoDB connection, same as the rest of the backend test suite — follow whatever test-database setup Member 3 establishes in Document 3 for consistency.)

14.2 Manual test checklist

- [ ] Trigger a real Brevo send to a real test inbox you control; confirm it arrives and reads correctly.

- [ ] Trigger a real Telegram message to a real linked chat; confirm it arrives.
- [ ] Run `POST /api/monitor/run` (or `runMonitorLocally.js`) twice in a row with an unchanged fixture price; confirm the second run produces no new `"sent"` NotificationEvent for the same alert (ACC-05).
- [ ] Temporarily set `forceFail: true` on a fixture offer an alert depends on, run a monitoring cycle, confirm `Offer.lastStatus` becomes `"error"`, `priceInPaise` is unchanged, and no alert fires (ACC-06). Set `forceFail` back to `false` afterward.
- [ ] Lower a fixture price below an alert's target (Section 6.3), run a cycle, confirm exactly one `"sent"` NotificationEvent per selected channel (ACC-04).

---

15. Debugging

Symptom	Likely cause	How to tell	Fix
Brevo returns `401`	Wrong/revoked `BREVO_API_KEY`	`err.response.status === 401`; body usually says something like "Key not found"	Regenerate the key in the Brevo dashboard, update `server/.env`, restart the server
Brevo returns `400`	Sender not verified, or malformed request	`err.response.status === 400`; body mentions the sender address or a validation field	Finish sender verification (Section 11, step 2) in the Brevo dashboard — a 400 here is almost always about the `sender.email`, not your API key
Telegram "chat not found"	The user hasn't pressed "Start" on the bot (or hasn't linked yet), or the stored `telegramChatId` is stale	Response body: `"description": "Bad Request: chat not found"`	Confirm the user completed `/start <code>` and the link-confirm step; a chat ID is only valid once the user has messaged the bot at least once
GitHub Actions workflow fails silently	Usually not actually silent — check the Actions tab	Repo → **Actions** tab → click the failed run → expand the "Call POST /api/monitor/run" step; the `curl` status line and response body are printed right there	If status is `401`: `MONITOR_TRIGGER_SECRET` mismatch between the GitHub secret and the deployed backend's env var. If status is a timeout/connection error: `BACKEND_URL` secret is wrong, or the backend is down
A provider function throws instead of returning `success:false`	A missing `try/catch`, or an assumption about the response shape that doesn't hold (e.g. `response.data.productBaseInfoV1` is `undefined` and something downstream dereferences a field on it without a check)	`priceMonitor.js`'s monitoring cycle crashes or skips remaining offers; check the server log for an unhandled exception stack trace pointing into `providers/`	Wrap the offending code in `try/catch` and return `{ success: false, errorMessage }` from the `catch` block — re-read Section 5.2: this contract is absolute, every provider function must honor it

---

16. Git workflow

Use exactly these branches (Document 1, Section 14; MASTER-PROMPT-SOURCE.md, Section 14) — do not invent new ones: `feature/retailer-provider`, `feature/price-monitor` (only for the thin interface-testing parts in Section 9.2 that touch this area — the real monitoring logic is Member 1's), `feature/email-notifications`, `feature/telegram-notifications`.

Full Git mechanics (cloning, resolving conflicts, PR review etiquette) are in Document 5 — this is the pattern specific to this document's work, shown once for `feature/retailer-provider`:

```
cd D:\Projects\PriceRadar
  git switch main
  git pull --ff-only origin main
  git switch -c feature/retailer-provider

  # ... implement providers/index.js, fixture.js, fixtureData.json, flipkart.js ...
  # ... run tests: cd server; npx jest tests/providers.*.test.js ...

  git status
  git diff
  git add server/src/providers server/tests/providers.dispatcher.test.js server/tests/p
  roviders.fixture.test.js
  git diff --cached
  git commit -m "feat: add provider dispatcher, fixture provider, and flipkart provider
  (not wired in)"
  git push -u origin feature/retailer-provider
```

Then open a pull request on GitHub, describe the change (note explicitly in the PR description: "Flipkart provider is built but NOT wired into the dispatcher — fixture only"), and request review from Member 1 before merging. Repeat this same pattern on separate branches for `feature/email-notifications` (Section 11) and `feature/telegram-notifications` (Section 12) — one focused branch and PR per feature area, not one giant branch for everything in this document.

---

17. Acceptance criteria

Tied to Document 1, Section 17:

ID	Check	This document's responsibility
ACC-01	Searching a known fixture product returns it with at least 2 labeled offers	`fixtureData.json` has 6 products, each with a Flipkart- and Amazon-labeled offer
ACC-04	A fixture price edited below an alert's target, followed by `POST /api/monitor/run`, results in exactly one `"sent"` `NotificationEvent` per channel	`sendAndRecordNotification` + the dedup logic (Section 13)
ACC-05	Calling `/api/monitor/run` again immediately, price unchanged, produces no new `"sent"` events	`dedupeKey` + unique index (Section 13.1)
ACC-06	A forced provider failure results in `Offer.lastStatus = "error"`, never a price of 0, never a false alert	Provider contract (Section 5.2) + the update pattern (Section 10.2)
ACC-08	The GitHub Actions workflow run log shows at least one successful `POST /api/monitor/run` call against the deployed backend	`.github/workflows/monitor.yml` (Section 8.2), verified once Document 5's deployment is live
ACC-10	Every `isSample: true` offer visibly shows a "Sample data" badge	`isSample: true` set correctly on every fixture entry (Section 6.1); UI badge itself is Member 2's responsibility, fed by this field

This document's own acceptance criteria:

- [ ] Every provider function (`fixture.js`, `flipkart.js`, the dispatcher) never throws under any tested failure condition — verified by Section 9.2's and Section 14's Jest tests.
- [ ] `flipkart.js` exists, is complete, and is clearly documented as **not wired into the dispatcher** unless and until the Section 7.4 switch-over checklist has actually been completed and verified against a real response.
- [ ] No code path anywhere attempts a live Amazon API call — verified by the dispatcher's explicit `errorMessage` for any non-sample Amazon offer (Section 5.3).
- [ ] The shared Brevo sender is verified and a real test email has been received.
- [ ] The Telegram bot responds to `/start <code>` and a real linked chat receives a real message.
- [ ] The `dedupeKey` format exactly matches Document 1, Section 9.3, and a manual re-trigger within the same hour does not produce a second send.